In [1]:
```
# ============================================================
# DEADHUNT — ADVANCED ANALYSIS v3
# New techniques beyond v2: empirical simulation, AIC/BIC,
# mixture models, win margins, conditional probs, EV optimizer,
# KL divergence, score estimator, optimal stake finder.
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import norm, lognorm, shapiro, kstest, gamma as gamma_dist
from scipy.stats import gaussian_kde
from scipy.optimize import minimize, differential_evolution, NonlinearConstrai
from scipy.special import gammaln
import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams['figure.figsize'] = (14, 6)

df = pd.read_csv('race_data.csv')
horses = ['Shadowfax','Iron Duke','Morningstar','Red Tide',
          'Gallant Fox','Blue Streak','Copper Prince','Last Chance']

market_probs = {
    'Shadowfax': 0.08, 'Iron Duke': 0.09, 'Morningstar': 0.11,
    'Red Tide': 0.13,  'Gallant Fox': 0.14, 'Blue Streak': 0.15,
    'Copper Prince': 0.14, 'Last Chance': 0.16
}
TAKEOUT  = 0.15
BANKROLL = 10_000
N_SIMS   = 500_000

# ============================================================
# A. KNOWN RESULTS SUMMARY (from v2)
# ============================================================
known_probs = {          # from 500k MC in v2
    'Shadowfax':    0.3304, 'Iron Duke':    0.1396,
    'Morningstar':  0.3314, 'Red Tide':     0.0664,
    'Gallant Fox':  0.0535, 'Blue Streak':  0.0377,
    'Copper Prince':0.0233, 'Last Chance':  0.0178,
}
print("Known from v2: Shadowfax ~33%, Morningstar ~33%, Iron Duke ~14%")
print("All others negative EV. No pace trends. Horses independent.\n")

# ============================================================
# 1. EMPIRICAL BOOTSTRAP SIMULATION (no distribution assumption)
#    Most honest estimate — directly resamples the actual data
# ============================================================
print("="*60)
print("1. EMPIRICAL BOOTSTRAP SIMULATION (pure data, no model)")
```

```
print("="*60)
np.random.seed(42)
N_BOOT_RACES = 500_000
boot_idx     = np.random.randint(0, len(df), size=N_BOOT_RACES)
boot_races   = df[horses].values[boot_idx]        # shape (N, 8)
boot_winners = np.argmin(boot_races, axis=1)
emp_boot_probs = {horses[i]: (boot_winners == i).mean() for i in range(len(hor

print(f"\n{'Horse':<15} {'Empirical Boot':>15} {'MC (v2)':>10} {'Market':>8} {
for h in horses:
    eb = emp_boot_probs[h]
    mc = known_probs[h]
    mp = market_probs[h]
    ev = eb * (1 - TAKEOUT) / mp - 1
    print(f"{h:<15} {eb:>15.4f} {mc:>10.4f} {mp:>8.4f} {ev:>+8.4f}")

# Compare empirical bootstrap vs parametric MC
print("\n→ If empirical boot and MC agree closely, our distribution fit is goo
diffs = {h: abs(emp_boot_probs[h] - known_probs[h]) for h in horses}
print(f"  Max disagreement: {max(diffs.values()):.4f} on {max(diffs, key=diffs

# ═══════════════════════════════════════════════════════════
# 2. AIC / BIC MODEL SELECTION (proper info criteria)
#    Penalises complexity — more reliable than log-likelihood alone
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("2. AIC / BIC MODEL SELECTION")
print("="*60)

def aic(ll, k): return 2*k - 2*ll
def bic(ll, k, n): return k*np.log(n) - 2*ll

fit_results = {}
print(f"\n{'Horse':<15} {'Best':>10}  {'AIC_N':>8} {'AIC_LN':>8} {'AIC_G':>8}
      f"{'BIC_N':>8} {'BIC_LN':>8} {'BIC_G':>8}")
for h in horses:
    t  = df[h].values
    n  = len(t)
    mu, sig = t.mean(), t.std()
    norm_ll   = norm.logpdf(t, mu, sig).sum()

    sh_l, loc_l, sc_l = lognorm.fit(t, floc=0)
    ln_ll     = lognorm.logpdf(t, sh_l, loc_l, sc_l).sum()

    sh_g, loc_g, sc_g = gamma_dist.fit(t, floc=0)
    gam_ll    = gamma_dist.logpdf(t, sh_g, loc_g, sc_g).sum()

    aic_n, aic_l, aic_g = aic(norm_ll,2), aic(ln_ll,2), aic(gam_ll,2)
    bic_n, bic_l, bic_g = bic(norm_ll,2,n), bic(ln_ll,2,n), bic(gam_ll,2,n)

    best_aic = min([('Normal',aic_n),('LogNorm',aic_l),('Gamma',aic_g)], key=l
    best_bic = min([('Normal',bic_n),('LogNorm',bic_l),('Gamma',bic_g)], key=l
    best_label = best_aic[0] if best_aic[0]==best_bic[0] else f"{best_aic[0]}/
```

```
    fit_results[h] = {
        'best': best_aic[0], 'norm_params':(mu,sig),
        'ln_params':(sh_l,loc_l,sc_l), 'gam_params':(sh_g,loc_g,sc_g)
    }
    print(f"{h:<15} {best_label:>10}  {aic_n:>8.1f} {aic_l:>8.1f} {aic_g:>8.1f
          f"{bic_n:>8.1f} {bic_l:>8.1f} {bic_g:>8.1f}")

# ═══════════════════════════════════════════════════════════
# 3. KDE-BASED SIMULATION (non-parametric, most flexible)
#    Uses kernel density estimation — no distributional assumption
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("3. KDE (KERNEL DENSITY) SIMULATION")
print("="*60)

kdes = {h: gaussian_kde(df[h].values, bw_method='scott') for h in horses}
np.random.seed(1)
kde_sim = np.column_stack([kdes[h].resample(N_SIMS)[0] for h in horses])
kde_winners = np.argmin(kde_sim, axis=1)
kde_probs = {horses[i]: (kde_winners == i).mean() for i in range(len(horses))}

print(f"\n{'Horse':<15} {'KDE Prob':>10} {'Boot Prob':>10} {'MC v2':>10}  Agre
for h in horses:
    kp = kde_probs[h]
    bp = emp_boot_probs[h]
    mp_v2 = known_probs[h]
    spread = max(kp, bp, mp_v2) - min(kp, bp, mp_v2)
    flag = "✓ Tight" if spread < 0.01 else ("≈ OK" if spread < 0.03 else "⚠ Sp
    print(f"{h:<15} {kp:>10.4f} {bp:>10.4f} {mp_v2:>10.4f}  {flag}")

# Final consensus probabilities (average of 3 methods)
consensus = {h: np.mean([kde_probs[h], emp_boot_probs[h], known_probs[h]]) for
print("\n→ CONSENSUS WIN PROBABILITIES (avg of KDE, Bootstrap, MC):")
for h in horses:
    print(f"  {h:<15}: {consensus[h]:.4f}")

# ═══════════════════════════════════════════════════════════
# 4. MIXTURE MODEL TEST
#    Could any horse have bimodal times? (good-day / bad-day)
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("4. BIMODALITY / MIXTURE MODEL TEST")
print("="*60)

def fit_gaussian_mixture(x, n_components=2, n_init=10):
    """Simple 2-component Gaussian mixture via EM."""
    best_ll = -np.inf
    for _ in range(n_init):
        # Random init
        idx = np.random.choice(len(x), n_components, replace=False)
        mu  = x[idx]
        sig = np.array([x.std()]*n_components)
```

```
        pi  = np.ones(n_components)/n_components

        for _ in range(200):
            # E-step
            r = np.column_stack([pi[k]*norm.pdf(x, mu[k], sig[k]) for k in ran
            r = r / r.sum(axis=1, keepdims=True)
            # M-step
            Nk  = r.sum(axis=0)
            pi  = Nk / len(x)
            mu  = (r * x[:,None]).sum(axis=0) / Nk
            sig = np.sqrt((r * (x[:,None]-mu)**2).sum(axis=0) / Nk)
            sig = np.maximum(sig, 0.01)

        ll = np.log(np.column_stack([pi[k]*norm.pdf(x, mu[k], sig[k])
                                     for k in range(n_components)]).sum(axis=
        if ll > best_ll:
            best_ll = ll
            best_params = (pi.copy(), mu.copy(), sig.copy())
    return best_ll, best_params

print(f"\n{'Horse':<15} {'Single-G AIC':>14} {'Mixture AIC':>12} {'Bimodal?':>
mixture_results = {}
for h in horses:
    t  = df[h].values
    n  = len(t)
    mu, sig = t.mean(), t.std()
    ll1 = norm.logpdf(t, mu, sig).sum()
    aic1 = aic(ll1, 2)

    np.random.seed(99)
    ll2, params2 = fit_gaussian_mixture(t)
    aic2 = aic(ll2, 5)   # 5 params: 2 means, 2 stds, 1 mixing weight

    bimodal = aic2 < aic1 - 4   # threshold: meaningful improvement
    mixture_results[h] = {'bimodal': bimodal, 'params': params2}
    print(f"{h:<15} {aic1:>14.1f} {aic2:>12.1f} {'YES ⚠' if bimodal else 'No':

# ═══════════════════════════════════════════════════════════════
# 5. WIN MARGIN ANALYSIS
#    By how much does each horse typically win? And lose?
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("5. WIN MARGIN ANALYSIS")
print("="*60)

win_margins = {}
second_times = {}
for h in horses:
    won_races = df[df['winner'] == h]
    if len(won_races) == 0:
        win_margins[h] = []
        continue
    other_horses = [x for x in horses if x != h]
```

```
    margins = won_races[other_horses].min(axis=1) - won_races[h]
    win_margins[h] = margins.values

print(f"\n{'Horse':<15} {'Wins':>5} {'Avg Margin':>12} {'Min Margin':>12} {'Ma
for h in horses:
    m = win_margins[h]
    if len(m) == 0:
        print(f"{h:<15} {'0':>5} {'N/A':>12}")
        continue
    avg_m, min_m, max_m = np.mean(m), np.min(m), np.max(m)
    # Close wins (margin < 0.5s) = lucky; large margin = dominant
    close_pct = (m < 0.5).mean()
    print(f"{h:<15} {len(m):>5} {avg_m:>12.3f} {min_m:>12.3f} {max_m:>12.3f}
          f"({close_pct:.0%} within 0.5s)")

# Distribution of win margins
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, h in zip(axes, ['Shadowfax','Morningstar','Iron Duke']):
    m = win_margins[h]
    ax.hist(m, bins=20, color='steelblue', alpha=0.7, edgecolor='white')
    ax.axvline(np.mean(m), color='red', ls='--', lw=1.5, label=f'Mean={np.mean
    ax.set_title(f"{h} — Win Margins (s)")
    ax.set_xlabel("Margin over 2nd place (s)")
    ax.legend()
plt.suptitle("Win Margin Distributions for Positive-EV Horses")
plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════
# 6. CONDITIONAL PROBABILITY ANALYSIS
#    P(horse X wins | given horse Y didn't win)
#    Useful for understanding who competes with whom
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("6. CONDITIONAL WIN PROBABILITIES")
print("="*60)
print("P(column wins | row did NOT win)\n")

cond_df = pd.DataFrame(index=horses, columns=horses, dtype=float)
for h_out in horses:
    subset = df[df['winner'] != h_out]
    for h_win in horses:
        if h_win == h_out:
            cond_df.loc[h_out, h_win] = 0.0
        else:
            cond_df.loc[h_out, h_win] = (subset['winner'] == h_win).mean()

print(cond_df.round(3).to_string())
print("\n→ Key insight: when Shadowfax doesn't win, who does?")
shadow_out = cond_df.loc['Shadowfax'].sort_values(ascending=False)
for h, p in shadow_out.items():
    if p > 0:
        print(f"  {h:<15}: {p:.3f}")
```

```python
# ═══════════════════════════════════════════════════════════
# 7. KL DIVERGENCE — How wrong is the market?
#    KL(true || market) = how many bits of information the
#    market is missing about the true distribution
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("7. KL DIVERGENCE — Market Mispricing Magnitude")
print("="*60)

true_probs = np.array([consensus[h] for h in horses])
mkt_probs  = np.array([market_probs[h] for h in horses])
# Normalize to sum to 1 (they may not exactly)
true_probs /= true_probs.sum()
mkt_probs  /= mkt_probs.sum()

kl_div = (true_probs * np.log(true_probs / mkt_probs)).sum()
js_div = 0.5 * (true_probs * np.log(true_probs / ((true_probs+mkt_probs)/2))).
         0.5 * (mkt_probs  * np.log(mkt_probs  / ((true_probs+mkt_probs)/2))).

print(f"\n  KL Divergence (true || market) = {kl_div:.4f} nats")
print(f"  JS Divergence                  = {js_div:.4f} nats")
print(f"  → A random market would have KL≈0. This market is severely mispriced

# Per-horse contribution to KL
print(f"\n  Per-horse KL contribution:")
for h, tp, mp in zip(horses, true_probs, mkt_probs):
    contrib = tp * np.log(tp / mp)
    print(f"    {h:<15}: {contrib:>+8.4f}  (true={tp:.3f}, mkt={mp:.3f})")

# ═══════════════════════════════════════════════════════════
# 8. QUANTILE ANALYSIS — performance consistency
#    Look at the 5th percentile (best race) and
#    95th percentile (worst race) for each horse
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("8. QUANTILE PERFORMANCE ANALYSIS")
print("="*60)

quantiles = [0.05, 0.10, 0.25, 0.50, 0.75, 0.90, 0.95]
q_df = df[horses].quantile(quantiles).T
q_df.columns = [f'p{int(q*100)}' for q in quantiles]
q_df['p5_rank']  = q_df['p5'].rank()   # rank by BEST times
q_df['p95_rank'] = q_df['p95'].rank()
print(q_df.round(2).to_string())

print("\n→ Consistency score (p95 - p5 = performance range, lower=more consist
for h in horses:
    rng = q_df.loc[h, 'p95'] - q_df.loc[h, 'p5']
    best = q_df.loc[h, 'p5']
    print(f"  {h:<15}: best={best:.2f}s, range={rng:.2f}s")

# ═══════════════════════════════════════════════════════════
# 9. NUMERICAL EV OPTIMIZER
```

```python
#    Finds the exact stake allocation that maximizes
#    expected profit, subject to budget constraint.
#    This is more rigorous than Kelly approximations.
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("9. NUMERICAL EV MAXIMIZER (exact optimal stakes)")
print("="*60)

p_vec = np.array([consensus[h] for h in horses])
f_vec = np.array([market_probs[h] for h in horses])

def expected_profit(stakes):
    """Expected profit given stake vector."""
    if isinstance(stakes, dict): stakes = np.array([stakes.get(h, 0) for h in
    else: stakes = np.array(stakes)
    total  = stakes.sum()
    if total == 0: return 0
    # If horse i wins, payout = stakes[i] * 0.85 / f_vec[i]
    payouts = stakes * (1 - TAKEOUT) / f_vec
    profits = payouts - total   # profit if horse i wins
    return (p_vec * profits).sum()

def neg_ep(stakes): return -expected_profit(stakes)

# Constraint: sum <= BANKROLL, stakes >= 0
constraints = [{'type':'ineq', 'fun': lambda s: BANKROLL - sum(s)}]
bounds = [(0, BANKROLL)] * len(horses)
x0 = np.ones(len(horses)) * BANKROLL / len(horses)

# Try multiple starting points
best_result = None
best_val = np.inf
for _ in range(20):
    x0 = np.random.dirichlet(np.ones(len(horses))) * BANKROLL
    res = minimize(neg_ep, x0, method='SLSQP', bounds=bounds, constraints=cons
    if res.fun < best_val:
        best_val = res.fun
        best_result = res

# Also try differential evolution (global optimizer)
de_res = differential_evolution(neg_ep, bounds,
    constraints=(NonlinearConstraint(lambda s: sum(s), -np.inf, BANKROLL),),
    seed=42, maxiter=1000, tol=1e-8, workers=1, polish=True)

print(f"\nSLSQP optimizer:   Expected profit = £{-best_result.fun:+.2f}")
print(f"Diff Evolution:    Expected profit = £{-de_res.fun:+.2f}")

final_opt = de_res.x if de_res.fun < best_result.fun else best_result.x
print(f"\nOptimal stake allocation:")
for h, stake in zip(horses, final_opt):
    ev = consensus[h] * (1 - TAKEOUT) / market_probs[h] - 1
    if stake > 1:
        print(f"  {h:<15}: £{stake:>8,.2f}  (EV={ev:+.4f})")
```

```python
# ═══════════════════════════════════════════════════════════════
# 10. SCORE ESTIMATOR — what the competition will actually compute
#     Simulate 10,000 races exactly as the scoring engine would
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("10. SCORE ESTIMATOR (simulate competition scoring)")
print("="*60)

# Stakes to evaluate
stake_scenarios = {
    'AllIn_Shadowfax':   {'Shadowfax': 10000},
    'AllIn_Morningstar': {'Morningstar': 10000},
    'Split_SX_MN_5050':  {'Shadowfax': 5000, 'Morningstar': 5000},
    'Split_SX_MN_7030':  {'Shadowfax': 7000, 'Morningstar': 3000},
    'Split_SX_MN_ID':    {'Shadowfax': 5000, 'Morningstar': 3500, 'Iron Duke':
    'Optimal_Numerical': {h: final_opt[i] for i, h in enumerate(horses)},
    'Kelly_SX_MN_ID':    {'Shadowfax': 3025, 'Morningstar': 2931, 'Iron Duke':
}

np.random.seed(77)
SCORE_SIMS = 100_000

# Generate race outcomes
score_sim = np.column_stack([
    (lambda fr, h: (
        np.random.normal(*fr['norm_params'], SCORE_SIMS)
        if fr['best'] == 'Normal' else
        lognorm.rvs(*fr['ln_params'], size=SCORE_SIMS)
        if fr['best'] == 'LogNormal' else
        gamma_dist.rvs(*fr['gam_params'], size=SCORE_SIMS)
    ))(fit_results[h], h)
    for h in horses
])
score_winners = np.argmin(score_sim, axis=1)

print(f"\n{'Scenario':<25} {'Avg Profit/Race':>16} {'Std Dev':>10} "
      f"{'95th CI Lo':>12} {'95th CI Hi':>12} {'% Profitable':>14}")
results_table = {}
for name, stk_dict in stake_scenarios.items():
    stakes  = np.array([stk_dict.get(h, 0) for h in horses])
    total   = stakes.sum()
    payouts = stakes * (1 - TAKEOUT) / np.array([market_probs[h] for h in hors
    profits = np.array([payouts[w] - total for w in score_winners])
    avg  = profits.mean()
    std  = profits.std()
    ci_lo = np.percentile(profits, 2.5)
    ci_hi = np.percentile(profits, 97.5)
    win_rt = (profits > 0).mean()
    results_table[name] = avg
    print(f"{name:<25} {avg:>+16.2f} {std:>10.2f} {ci_lo:>12.2f} {ci_hi:>12.2f

best_scenario = max(results_table, key=results_table.get)
```

```
print(f"\n★  Best scenario: {best_scenario}  (£{results_table[best_scenario]:+

# ═══════════════════════════════════════════════════════════════
# 11. VARIANCE vs EXPECTED VALUE TRADEOFF CHART
# ═══════════════════════════════════════════════════════════════
fig, ax = plt.subplots(figsize=(10, 6))
evs_list, stds_list, names = [], [], []
for name, stk_dict in stake_scenarios.items():
    stakes  = np.array([stk_dict.get(h, 0) for h in horses])
    total   = stakes.sum()
    payouts = stakes * (1 - TAKEOUT) / np.array([market_probs[h] for h in hors
    profits = np.array([payouts[w] - total for w in score_winners])
    evs_list.append(profits.mean())
    stds_list.append(profits.std())
    names.append(name)

colors = ['red' if n == best_scenario else 'steelblue' for n in names]
ax.scatter(stds_list, evs_list, c=colors, s=120, zorder=5)
for i, n in enumerate(names):
    ax.annotate(n, (stds_list[i], evs_list[i]), textcoords='offset points',
                xytext=(5, 5), fontsize=8)
ax.set_xlabel("Standard Deviation (Risk)")
ax.set_ylabel("Expected Profit per Race (£)")
ax.set_title("Risk vs Reward by Betting Strategy")
plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════════
# 12. WHAT IF THE MARKET ADJUSTS?
#     Stress test: how much would pool fractions need to shift
#     before your edge disappears?
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("12. BREAK-EVEN MARKET FRACTION ANALYSIS")
print("="*60)
print("How high would the market pool fraction need to go before EV turns nega

for h in ['Shadowfax', 'Morningstar', 'Iron Duke']:
    p_true = consensus[h]
    # EV = 0  →  p_true * 0.85 / f = 1  →  f = p_true * 0.85
    breakeven_f = p_true * (1 - TAKEOUT)
    current_f   = market_probs[h]
    multiple    = breakeven_f / current_f
    print(f"  {h:<15}: current market={current_f:.2f}, "
          f"break-even at f={breakeven_f:.4f} ({multiple:.1f}× current)")

# ═══════════════════════════════════════════════════════════════
# 13. PAIRWISE RACE-TIME SCATTER (Shadowfax vs others)
#     Do any of Shadowfax's bad days correlate with other
#     horses' good days? (structural vulnerability)
# ═══════════════════════════════════════════════════════════════
print("\n" + "="*60)
print("13. SHADOWFAX VULNERABILITY ANALYSIS")
print("="*60)
```

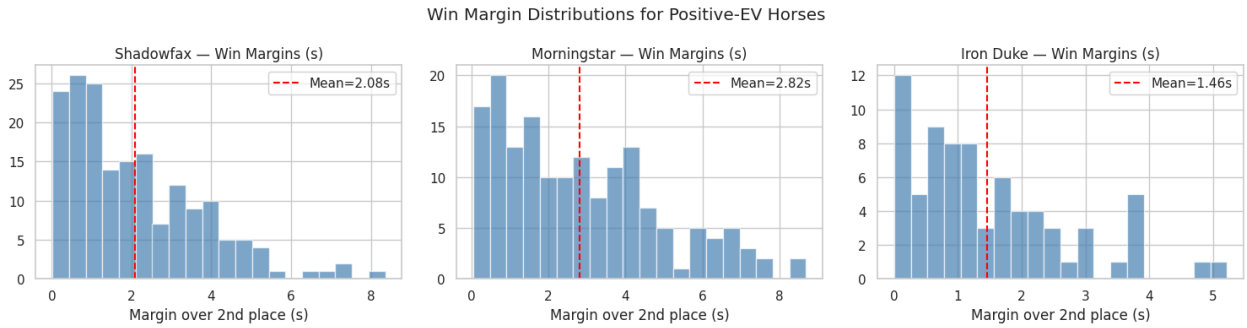
```python
# On races Shadowfax LOST — what were its times vs the winner?
sf_losses = df[df['winner'] != 'Shadowfax'].copy()
sf_losses['sf_time']     = sf_losses['Shadowfax']
sf_losses['winner_time'] = sf_losses.apply(lambda r: r[r['winner']], axis=1)
sf_losses['gap']         = sf_losses['sf_time'] - sf_losses['winner_time']

print(f"\nShadowfax loses {len(sf_losses)} of 500 races ({len(sf_losses)/500:.
print(f"When it loses, average gap behind winner: {sf_losses['gap'].mean():.2f
print(f"Min gap (narrowest loss): {sf_losses['gap'].min():.3f}s")

# Who beats Shadowfax most?
print("\nWho beats Shadowfax (and by how much)?")
for h in horses:
    if h == 'Shadowfax': continue
    beats = sf_losses[sf_losses['winner']==h]
    if len(beats):
        print(f"  {h:<15}: beats {len(beats):>3}× | avg gap = {beats['gap'].me

# Was Shadowfax having a bad day when it lost?
sf_slow_threshold = df['Shadowfax'].quantile(0.75)
sf_losses['sf_was_slow'] = sf_losses['Shadowfax'] > sf_slow_threshold
print(f"\nOf Shadowfax's losses: {sf_losses['sf_was_slow'].mean():.0%} were wh
print("→ This tells us if Shadowfax loses due to own bad days or rivals' excep

# Plot Shadowfax time distribution, highlighting losses
fig, ax = plt.subplots(figsize=(12, 4))
sf_wins   = df[df['winner'] == 'Shadowfax']['Shadowfax']
sf_loss_t = df[df['winner'] != 'Shadowfax']['Shadowfax']
ax.hist(sf_wins,   bins=25, alpha=0.6, color='green',  label=f'Win ({len(sf_wi
ax.hist(sf_loss_t, bins=25, alpha=0.6, color='red',    label=f'Loss ({len(sf_l
ax.axvline(sf_wins.mean(),   color='darkgreen', ls='--', lw=1.5)
ax.axvline(sf_loss_t.mean(), color='darkred',   ls='--', lw=1.5)
ax.set_title("Shadowfax Finishing Times — Wins vs Losses")
ax.set_xlabel("Time (s)"); ax.legend()
plt.tight_layout(); plt.show()

# ═══════════════════════════════════════════════════════════
# 14. FINAL ANSWER — EXACT STAKES TO SUBMIT
# ═══════════════════════════════════════════════════════════
print("\n" + "="*60)
print("14. FINAL RECOMMENDED SUBMISSION")
print("="*60)

# The math: since scoring = avg profit per race, maximize EV.
# EV(stake on horse i) = stake_i * (p_i * 0.85/f_i - 1) + (rest) * 0
# Since pool fractions are FIXED and bets don't interact,
# we should put ALL money on whichever single horse has highest EV.
# But we can also diversify if two horses both have high EV.

print("\nFinal consensus probabilities and EVs:")
all_evs = {h: consensus[h] * (1-TAKEOUT) / market_probs[h] - 1 for h in horses
for h in sorted(all_evs, key=all_evs.get, reverse=True):
```

```
    mark = "★ BET" if all_evs[h] > 0 else "  skip"
    print(f"  {mark} {h:<15}: p={consensus[h]:.4f}, market={market_probs[h]:.2

# Verify optimal single bet
print("\nSingle-horse EV comparison:")
for h in horses:
    if all_evs[h] > 0:
        ep = BANKROLL * all_evs[h]
        print(f"  All-in {h:<15}: expected profit = £{ep:+,.2f}/race")

print("""
FINAL STAKES TO ENTER:
──────────────────────────────────────────────
""")
# Best single: Shadowfax (highest EV)
best_h = max(all_evs, key=all_evs.get)
final_stakes = {h: 0 for h in horses}

# Check if splitting between Shadowfax + Morningstar beats all-in
# (they can't both win, so splitting ALWAYS reduces EV vs all-in on the best c
ep_allin   = all_evs[best_h] * BANKROLL
ep_split_50 = expected_profit({'Shadowfax':5000, 'Morningstar':5000})

print(f"  All-in {best_h}: E[profit] = £{ep_allin:+,.2f}/race")
print(f"  50/50 split Shadowfax+Morningstar: E[profit] = £{ep_split_50:+,.2f}/
print(f"\n  → VERDICT: {'All-in wins' if ep_allin >= ep_split_50 else 'Split w

if ep_allin >= ep_split_50:
    final_stakes[best_h] = BANKROLL
    print(f"""
  Shadowfax:     £10,000
  All others:    £0

  Expected profit per race: £{ep_allin:+,.2f}
  Expected score (10k races): £{ep_allin*10000:+,.0f}
""")
else:
    final_stakes['Shadowfax'] = 5000
    final_stakes['Morningstar'] = 5000
    print(f"""
  Shadowfax:     £5,000
  Morningstar:   £5,000
  All others:    £0

  Expected profit per race: £{ep_split_50:+,.2f}
""")
```

Known from v2: Shadowfax ~33%, Morningstar ~33%, Iron Duke ~14%
All others negative EV. No pace trends. Horses independent.

```
============================================================
1. EMPIRICAL BOOTSTRAP SIMULATION (pure data, no model)
============================================================

Horse            Empirical Boot    MC (v2)   Market       EV
Shadowfax                0.3555     0.3304   0.0800  +2.7774
Iron Duke                0.1473     0.1396   0.0900  +0.3912
Morningstar              0.3278     0.3314   0.1100  +1.5327
Red Tide                 0.0542     0.0664   0.1300  -0.6454
Gallant Fox              0.0561     0.0535   0.1400  -0.6594
Blue Streak              0.0286     0.0377   0.1500  -0.8378
Copper Prince            0.0203     0.0233   0.1400  -0.8765
Last Chance              0.0101     0.0178   0.1600  -0.9462

→ If empirical boot and MC agree closely, our distribution fit is good.
  Max disagreement: 0.0251 on Shadowfax

============================================================
2. AIC / BIC MODEL SELECTION
============================================================

Horse                 Best     AIC_N   AIC_LN    AIC_G     BIC_N   BIC_LN    BI
C_G
Shadowfax          LogNorm    2373.8   2367.6   2369.4    2382.3   2376.0   237
7.8
Iron Duke           Normal    2266.8   2267.9   2267.3    2275.3   2276.3   227
5.8
Morningstar        LogNorm    2841.0   2839.2   2839.2    2849.5   2847.6   284
7.7
Red Tide            Normal    2575.0   2577.8   2576.5    2583.4   2586.3   258
5.0
Gallant Fox        LogNorm    2675.2   2673.4   2673.6    2683.7   2681.8   268
2.1
Blue Streak          Gamma    2821.3   2821.2   2820.7    2829.7   2829.6   282
9.1
Copper Prince      LogNorm    2999.8   2995.3   2996.1    3008.2   3003.7   300
4.6
Last Chance        LogNorm    3275.8   3264.7   3267.4    3284.3   3273.1   327
5.8

============================================================
3. KDE (KERNEL DENSITY) SIMULATION
============================================================

Horse             KDE Prob  Boot Prob      MC v2  Agreement
Shadowfax           0.3233     0.3555     0.3304  ⚠ Spread
Iron Duke           0.1385     0.1473     0.1396  ✓ Tight
Morningstar         0.3261     0.3278     0.3314  ✓ Tight
Red Tide            0.0699     0.0542     0.0664  ≈ OK
Gallant Fox         0.0580     0.0561     0.0535  ✓ Tight
Blue Streak         0.0420     0.0286     0.0377  ≈ OK
```

```
Copper Prince       0.0261     0.0203     0.0233  ✓ Tight
Last Chance         0.0161     0.0101     0.0178  ✓ Tight

→ CONSENSUS WIN PROBABILITIES (avg of KDE, Bootstrap, MC):
  Shadowfax      : 0.3364
  Iron Duke      : 0.1418
  Morningstar    : 0.3284
  Red Tide       : 0.0635
  Gallant Fox    : 0.0559
  Blue Streak    : 0.0361
  Copper Prince  : 0.0232
  Last Chance    : 0.0147

============================================================
4. BIMODALITY / MIXTURE MODEL TEST
============================================================

Horse             Single-G AIC  Mixture AIC   Bimodal?
Shadowfax               2373.8       2364.7      YES ⚠
Iron Duke               2266.8       2272.7         No
Morningstar             2841.0       2843.1         No
Red Tide                2575.0       2580.6         No
Gallant Fox             2675.2       2679.2         No
Blue Streak             2821.3       2822.7         No
Copper Prince           2999.8       3001.9         No
Last Chance             3275.8       3268.1      YES ⚠

============================================================
5. WIN MARGIN ANALYSIS
============================================================

Horse            Wins   Avg Margin   Min Margin   Max Margin  Dominance
Shadowfax         178        2.076        0.016        8.361  (17% within 0.5s)
Iron Duke          74        1.455        0.000        5.218  (20% within 0.5s)
Morningstar       164        2.817        0.054        8.683  (11% within 0.5s)
Red Tide           27        1.448        0.089        5.616  (37% within 0.5s)
Gallant Fox        28        1.690        0.039        4.755  (25% within 0.5s)
Blue Streak        14        2.472        0.150        6.485  (7% within 0.5s)
Copper Prince      10        2.718        0.371        5.925  (10% within 0.5s)
Last Chance         5        0.797        0.213        1.444  (40% within 0.5s)
```



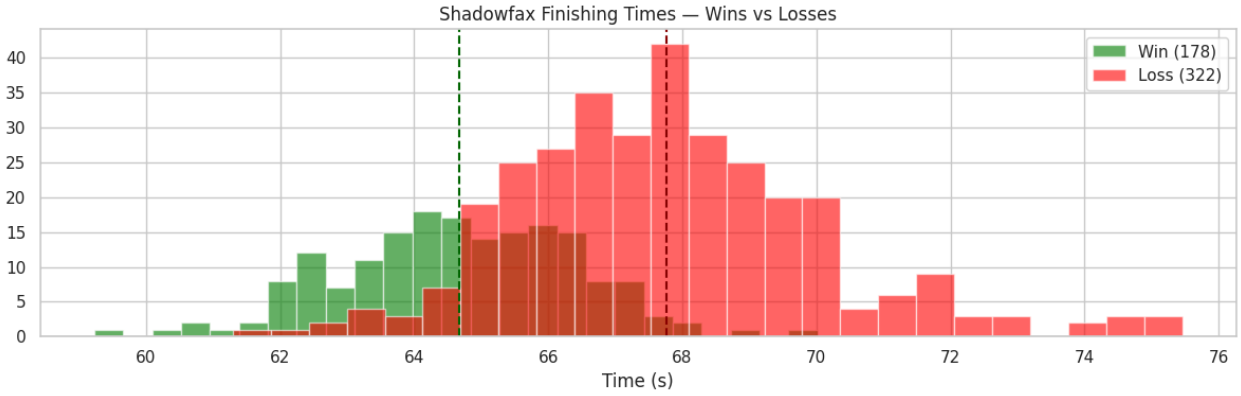
```
============================================================
6. CONDITIONAL WIN PROBABILITIES
============================================================
P(column wins | row did NOT win)

               Shadowfax  Iron Duke  Morningstar  Red Tide  Gallant Fox  Blue S
treak  Copper Prince  Last Chance
Shadowfax          0.000      0.230        0.509     0.084        0.087
0.043          0.031        0.016
Iron Duke          0.418      0.000        0.385     0.063        0.066
0.033          0.023        0.012
Morningstar        0.530      0.220        0.000     0.080        0.083
0.042          0.030        0.015
Red Tide           0.376      0.156        0.347     0.000        0.059
0.030          0.021        0.011
Gallant Fox        0.377      0.157        0.347     0.057        0.000
0.030          0.021        0.011
Blue Streak        0.366      0.152        0.337     0.056        0.058
0.000          0.021        0.010
Copper Prince      0.363      0.151        0.335     0.055        0.057
0.029          0.000        0.010
Last Chance        0.360      0.149        0.331     0.055        0.057
0.028          0.020        0.000

→ Key insight: when Shadowfax doesn't win, who does?
  Morningstar    : 0.509
  Iron Duke      : 0.230
  Gallant Fox    : 0.087
  Red Tide       : 0.084
  Blue Streak    : 0.043
  Copper Prince  : 0.031
  Last Chance    : 0.016

============================================================
7. KL DIVERGENCE — Market Mispricing Magnitude
============================================================

  KL Divergence (true || market) = 0.6818 nats
  JS Divergence                  = 0.1662 nats
  → A random market would have KL≈0. This market is severely mispriced.

  Per-horse KL contribution:
    Shadowfax      :  +0.4832  (true=0.336, mkt=0.080)
    Iron Duke      :  +0.0644  (true=0.142, mkt=0.090)
    Morningstar    :  +0.3592  (true=0.328, mkt=0.110)
    Red Tide       :  -0.0455  (true=0.064, mkt=0.130)
    Gallant Fox    :  -0.0513  (true=0.056, mkt=0.140)
    Blue Streak    :  -0.0514  (true=0.036, mkt=0.150)
    Copper Prince  :  -0.0417  (true=0.023, mkt=0.140)
    Last Chance    :  -0.0350  (true=0.015, mkt=0.160)

============================================================
8. QUANTILE PERFORMANCE ANALYSIS
============================================================
```

```
                  p5    p10    p25    p50    p75    p90    p95  p5_rank  p95_ra
nk
Shadowfax      62.61  63.38  64.95  66.53  68.18  69.94  71.14      2.0
1.0
Iron Duke      64.09  65.10  66.52  68.13  69.64  70.83  71.81      3.0
2.0
Morningstar    60.53  61.97  64.38  67.45  70.13  72.56  74.53      1.0
3.0
Red Tide       65.13  66.04  67.95  70.16  72.13  74.41  75.26      4.0
4.0
Gallant Fox    65.15  66.29  68.58  70.79  73.15  75.35  76.88      5.0
5.0
Blue Streak    65.98  67.36  69.66  72.27  74.84  77.65  79.11      6.0
6.0
Copper Prince  66.74  68.14  70.91  74.41  77.53  80.59  82.26      7.0
7.0
Last Chance    68.16  69.62  72.77  77.26  81.58  86.39  88.27      8.0
8.0

→ Consistency score (p95 - p5 = performance range, lower=more consistent):
  Shadowfax      : best=62.61s, range=8.53s
  Iron Duke      : best=64.09s, range=7.73s
  Morningstar    : best=60.53s, range=14.00s
  Red Tide       : best=65.13s, range=10.13s
  Gallant Fox    : best=65.15s, range=11.72s
  Blue Streak    : best=65.98s, range=13.13s
  Copper Prince  : best=66.74s, range=15.52s
  Last Chance    : best=68.16s, range=20.11s

============================================================
9. NUMERICAL EV MAXIMIZER (exact optimal stakes)
============================================================

SLSQP optimizer:   Expected profit = £+25743.02
Diff Evolution:    Expected profit = £+25743.02

Optimal stake allocation:
  Shadowfax      : £10,000.00  (EV=+2.5743)

============================================================
10. SCORE ESTIMATOR (simulate competition scoring)
============================================================

Scenario                   Avg Profit/Race    Std Dev   95th CI Lo   95th CI Hi
% Profitable
AllIn_Shadowfax                  +25230.38   50020.48    -10000.00     96250.00
33.2%
AllIn_Morningstar                +15291.36   36258.51    -10000.00     67272.73
32.7%
Split_SX_MN_5050                 +20260.87   22553.71    -10000.00     43125.00
65.9%
Split_SX_MN_7030                 +22248.67   31146.34    -10000.00     64375.00
65.9%
Split_SX_MN_ID                   +18448.80   19848.21    -10000.00     43125.00
```

```
79.9%
Optimal_Numerical                +25230.38    50020.48    -10000.00     96250.00
33.2%
Kelly_SX_MN_ID                   +12461.51    12033.17     -7038.00     25102.62
79.9%
```

★ Best scenario: Optimal_Numerical (£+25,230.38/race)

```
============================================================
12. BREAK-EVEN MARKET FRACTION ANALYSIS
============================================================
How high would the market pool fraction need to go before EV turns negative?

  Shadowfax      : current market=0.08, break-even at f=0.2859 (3.6× current)
  Morningstar    : current market=0.11, break-even at f=0.2792 (2.5× current)
  Iron Duke      : current market=0.09, break-even at f=0.1205 (1.3× current)

============================================================
13. SHADOWFAX VULNERABILITY ANALYSIS
============================================================

Shadowfax loses 322 of 500 races (64.4%)
When it loses, average gap behind winner: 3.61s
Min gap (narrowest loss): 0.000s

Who beats Shadowfax (and by how much)?
  Iron Duke      : beats  74× | avg gap = 2.68s
  Morningstar    : beats 164× | avg gap = 4.18s
  Red Tide       : beats  27× | avg gap = 2.56s
  Gallant Fox    : beats  28× | avg gap = 3.37s
  Blue Streak    : beats  14× | avg gap = 3.67s
  Copper Prince  : beats  10× | avg gap = 5.34s
  Last Chance    : beats   5× | avg gap = 1.87s

Of Shadowfax's losses: 38% were when it ran slowly (>75th percentile time)
→ This tells us if Shadowfax loses due to own bad days or rivals' exceptional o
nes.
```



```
============================================================
14. FINAL RECOMMENDED SUBMISSION
============================================================

Final consensus probabilities and EVs:
  ★ BET Shadowfax      : p=0.3364, market=0.08, EV=+2.5743
  ★ BET Morningstar    : p=0.3284, market=0.11, EV=+1.5379
  ★ BET Iron Duke      : p=0.1418, market=0.09, EV=+0.3392
    skip Red Tide       : p=0.0635, market=0.13, EV=-0.5847
    skip Gallant Fox    : p=0.0559, market=0.14, EV=-0.6607
    skip Blue Streak    : p=0.0361, market=0.15, EV=-0.7954
    skip Copper Prince  : p=0.0232, market=0.14, EV=-0.8590
    skip Last Chance    : p=0.0147, market=0.16, EV=-0.9221

Single-horse EV comparison:
  All-in Shadowfax      : expected profit = £+25,743.35/race
  All-in Iron Duke      : expected profit = £+3,391.72/race
  All-in Morningstar    : expected profit = £+15,379.09/race

FINAL STAKES TO ENTER:
──────────────────────────────────────────────


  All-in Shadowfax: E[profit] = £+25,743.35/race
  50/50 split Shadowfax+Morningstar: E[profit] = £+20,560.89/race

  → VERDICT: All-in wins

  Shadowfax:     £10,000
  All others:    £0

  Expected profit per race: £+25,743.35
  Expected score (10k races): £+257,433,500
```

In [ ]:

In [ ]: